

```
In [ ]: # data sets
# from torch_geometric.datasets import QM9 # we will not be using QM9
from tdc.utils import get_reaction_type

# rdkit things
from rdkit import Chem
from rdkit.Chem import GetPeriodicTable
from rdkit.Chem import rdChemReactions
from rdkit.Chem import Draw
from rdkit.Chem.Draw import ReactionToImage
from rdkit.Chem.Draw import MolToImage

# standard libraries
import glob
import os
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# machine learning libraries
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import GCNConv, global_mean_pool
from torch_geometric.data import Data
from torch_geometric.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
```

(1) Retrieve the cleaned data from data_exploration.ipynb to use in our model. Structure it to use in model.

```
In [2]: alchemy_elements = {'C', 'Cl', 'F', 'H', 'N', 'O', 'S'}
molecule_types_df = pd.read_csv("Collected Data/Alchemy-v20191129/final_versions.csv")
reaction_types_df = pd.read_csv("Collected Data/uspto_50k_processed.csv")

display(molecule_types_df)
display(reaction_types_df)
```

	gdb_idx	atom number	zpve\n(Ha, zero point vibrational energy)	Cv\n(cal/molK, heat capacity at 298.15 K)	gap\n(Ha, LUMO- HOMO)	G\n(Ha, Free energy at 298.15 K)	HOMO ene H
0	2859833	9	0.226164	0.000067	0.300531	-352.128828	-0.2
1	3148292	9	0.180782	0.000058	0.293098	-386.820766	-0.2
2	3607838	9	0.205922	0.000057	0.307544	-388.062328	-0.2
3	9540153	11	0.160845	0.000056	0.195923	-515.046508	-0.2
4	340363	10	0.177712	0.000057	0.254495	-403.843164	-0.2
...	...	...	...	...	...	...	...
202574	9581016	10	0.120222	0.000051	0.179768	-450.915828	-0.2
202575	9706527	10	0.130179	0.000053	0.189401	-438.632624	-0.2
202576	9760179	10	0.119919	0.000051	0.163827	-450.920199	-0.2
202577	9845175	10	0.119740	0.000050	0.191354	-454.722866	-0.2
202578	9880714	10	0.142321	0.000056	0.174621	-418.749178	-0.2

202579 rows × 15 columns

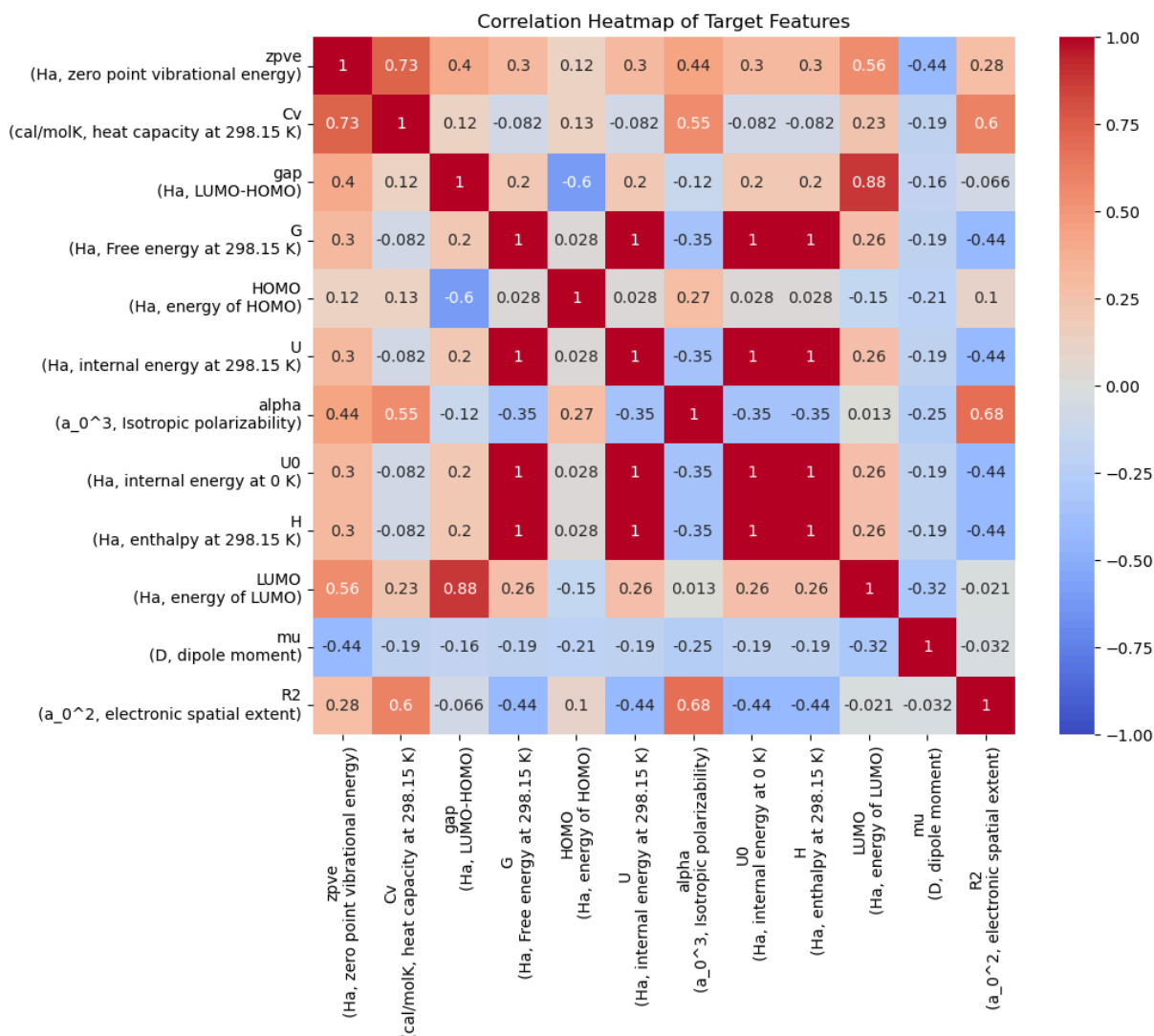
reactant		
0	<chem>C1=COCCC1.COC(=O)CCC(=O)c1ccc(O)cc1O</chem>	<chem>COC(=O)CCC(=O)c1ccc(OC2C</chem>
1	<chem>COC(=O)c1cccc(C(=O)O)c1.Nc1cccnc1N</chem>	<chem>COC(=O)c1cccc(-c2nc3ccc</chem>
2	<chem>CC(C)(C)OC(=O)NC1CCC(C(=O)O)CC1.CNOC</chem>	<chem>CON(C)C(=O)C1CCC(N</chem>
3	<chem>Nc1ccc(O)cc1.O=[N+](O-)]c1ccc(Cl)nc1Cl</chem>	<chem>O=[N+](O-)]c1ccc(Cl)nc1N</chem>
4	<chem>[N-]=[N+]=NCC1=CC[C@@H](c2ccc(Cl)cc2Cl)[C@H](...</chem>	<chem>NCC1=CC[C@@H](c2ccc(Cl)cc2Cl)[C@H](...</chem>
...	...	...
35111	<chem>COC(=O)c1ccc(C(=O)OCc2ccccc2)cc1F</chem>	<chem>COC(=O)c1ccc(C(=O)OCc2ccccc2)cc1F</chem>
35112	<chem>C=CCOC(=O)N1CCc2nnc(NN)cc2C1.CC(C)=O</chem>	<chem>C=CCOC(=O)N1CCc2nnc(NN)cc2C1.CC(C)=O</chem>
35113	<chem>FC1(F)CCNC1.O=C(O)c1ccncc1NC(=O)c1nc(C2CC2)ccc...</chem>	<chem>O=C(Nc1cnccc1C(=O)F)C1c1nc(C2CC2)ccc...</chem>
35114	<chem>CC(=O)Cl.CC(C)(C)OC(=O)NCCO</chem>	<chem>CC(=O)OCCNC(=O)CC(C)(C)OC(=O)NCCO</chem>
35115	<chem>CC(C)(C)O.O=CC1=C[C@H]2CC(O)O[C@H]2C1</chem>	<chem>(C)OC1C[C@@H]2C=C(C(=O)O)C1</chem>

35116 rows × 3 columns

The GNN will take each molecule as a pooling of the atoms as nodes and the bonds as edges and lets each atom update its features by aggregating from its neighbors. After this messaging, pool atoms to molecule to predict the target features that will be used in the decision tree for the reaction type. Now, for the decision tree that takes in these 12 output target features, I don't want redundancy. So I will remove the ones that have overlap.

```
In [3]: # just the target features for the molecules, which we will be trying to pre
# 12/15 are valid physics values for this model (smiles is already the graph
molecule_targets_df = molecule_types_df.drop(columns=['smiles', 'gdb_idx', '

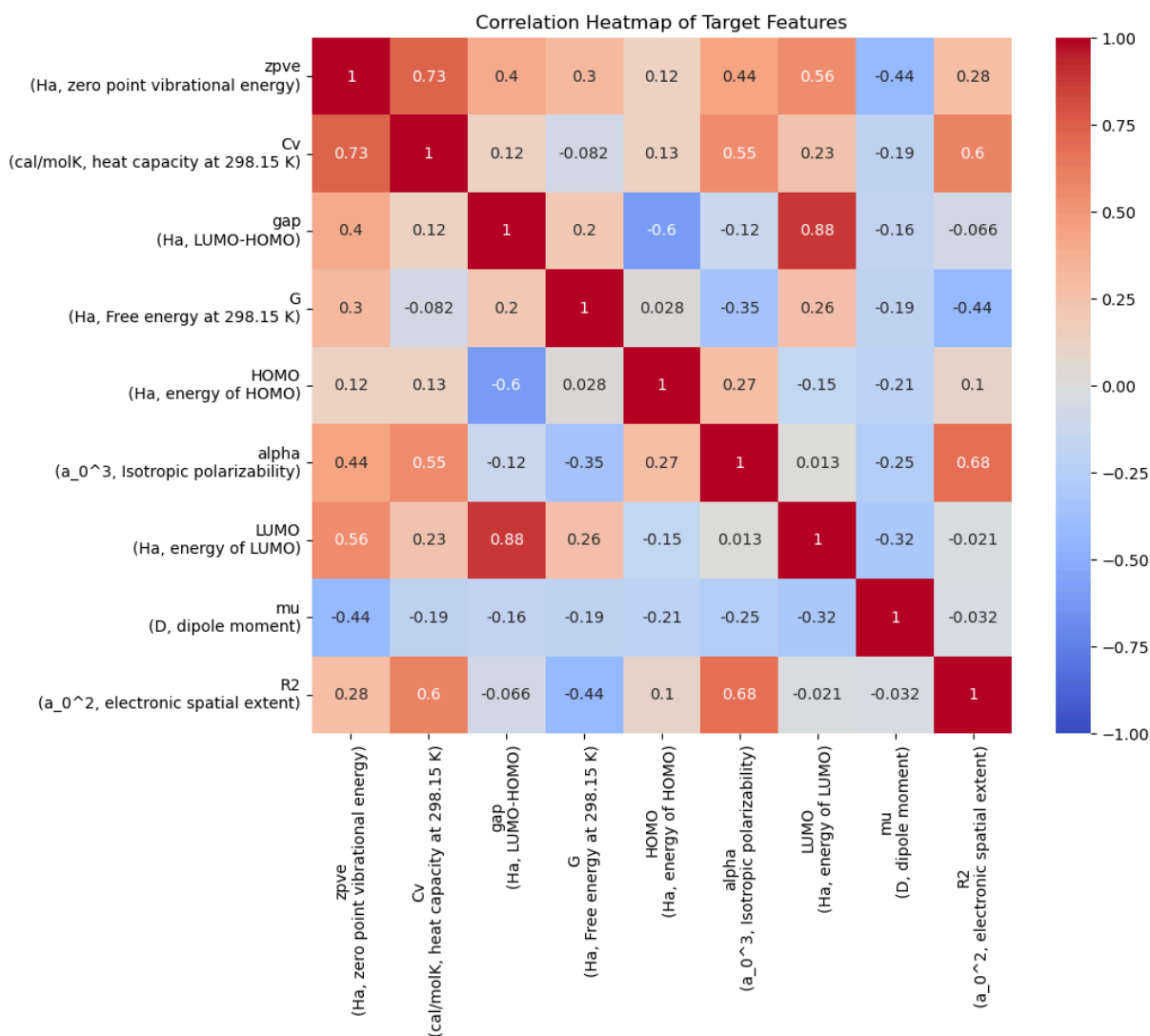
# correlation heatmap of the target features
plt.figure(figsize=(10, 8))
sns.heatmap(molecule_targets_df.corr(), annot=True, cmap='coolwarm', vmin=-1
plt.title("Correlation Heatmap of Target Features")
plt.show()
```



Of the 12, some of the thermodynamic ones seem redundant, so let's cut those and not waste time in the GNN calculating them cause they're not going in the decision tree. There will be nine remaining.

```
In [4]: # G, U, U0, H
# Thermo:  $G = U + PV - TS = H - TS$ , so we only really need one of these.
# Let's keep G because all the chemists love G
molecule_targets_df = molecule_types_df.drop(columns=[
    'smiles',
    'gdb_idx',
    'atom number',
    'U\n(Ha, internal energy at 298.15 K)',
    'U0\n(Ha, internal energy at 0 K)',
    'H\n(Ha, enthalpy at 298.15 K)',
])

# correlation heatmap of the target features
plt.figure(figsize=(10, 8))
sns.heatmap(molecule_targets_df.corr(), annot=True, cmap='coolwarm', vmin=-1
plt.title("Correlation Heatmap of Target Features")
plt.show()
```



(2) Focus on making the GNN with the alchemy DFT data. Use only a fraction of the data so I can practically run this.

```
In [5]: # Create a smaller dataframe with a random sample of molecules from the alch
n_examples = 5000
small_molecule_types_df = molecule_types_df.sample(n=n_examples, random_stat
display(small_molecule_types_df)
```

	gdb_idx	atom number	zpve\n(Ha, zero point vibrational energy)	Cv\n(cal/molK, heat capacity at 298.15 K)	gap\n(Ha, LUMO-HOMO)	G\n(Ha, Free energy at 298.15 K)	HOMO\n(Ha, energy HOF)
0	82642	11	0.239593	0.000072	0.250232	-502.508749	-0.229
1	6264428	10	0.211166	0.000063	0.253778	-463.251265	-0.233
2	8761235	10	0.188067	0.000058	0.210123	-462.067952	-0.224
3	6640342	10	0.181395	0.000065	0.305293	-859.209539	-0.249
4	153469	11	0.183893	0.000060	0.183995	-479.110377	-0.202
...	...	...	...	...	...	...	...
4995	5769183	12	0.174097	0.000068	0.182649	-570.272903	-0.254
4996	6487574	11	0.224063	0.000079	0.126059	-535.119847	-0.177
4997	2381774	10	0.126333	0.000055	0.203667	-761.533169	-0.231
4998	6572789	9	0.199144	0.000070	0.133468	-441.800727	-0.184
4999	912535	9	0.180015	0.000061	0.297655	-386.813872	-0.246

5000 rows x 15 columns

NOTE: I made this before the GNN tutorial in discussion (Tutorial 11) so I did it my way using GCNConv. When I finally clean up the code in the end, I will be more explicit like in the tutorial defining that graphs, but for now this is sufficient.

```
In [6]: def smiles_to_graph(smi, y):
        """
        Convert a SMILES string to a PyTorch Geometric Data object representing
        Nodes represent atoms, and edges represent bonds.
        """
        mol = Chem.MolFromSmiles(smi)
        if mol is None: return None

        # atom features
        atom_features = [atom.GetAtomicNum() for atom in mol.GetAtoms()]
        x = torch.tensor(atom_features, dtype=torch.float).unsqueeze(1)

        # edges
        edge_index = []
        for bond in mol.GetBonds():
```

```

        i = bond.GetBeginAtomIdx()
        j = bond.GetEndAtomIdx()
        edge_index.append((i, j))
        edge_index.append((j, i))
        # undirected graph, both directions

    # convert to tensor
    if len(edge_index) == 0:
        edge_index = torch.empty((2, 0), dtype=torch.long)
    else:
        edge_index = torch.tensor(edge_index, dtype=torch.long).t().contiguous()

    # targets
    y = torch.tensor(y, dtype=torch.float).unsqueeze(0)

    return Data(x=x, edge_index=edge_index, y=y) # already is a class that c

```

```

In [7]: # turn all the data into a graph for use in the GNN
        # molecule_targets_df is the target columns

graph_dataset = []
for idx, row in small_molecule_types_df.iterrows():
    graph = smiles_to_graph(
        row['smiles'],
        row[molecule_targets_df.columns].astype(float).values
    )
    if graph is not None:
        graph_dataset.append(graph)

g = graph_dataset[0]
print(g.x.shape)           # [num_atoms, 1]
print(g.edge_index.shape)  # [2, num_edges]
print(g.y.shape)           # should be 9
# okay great nothing exploded

```

```

torch.Size([11, 1])
torch.Size([2, 22])
torch.Size([1, 9])

```

```

In [8]: # normalize the targets

all_y = torch.stack([g.y for g in graph_dataset])
y_mean = all_y.mean(dim = 0)
y_std = all_y.std(dim = 0).clamp_min(1e-8) # avoid div by 0 error

for g in graph_dataset:
    g.y = (g.y - y_mean) / y_std

```

```

In [9]: class GNN(torch.nn.Module):
        # screw around with the hidden_dim to get better regression statistic
        # number of y predictions is out_dim in case of future change
        def __init__(self, in_channels=1, hidden_dim=128, out_dim = molecule_tar
            super().__init__()

        # A larger graph model with three graph convolution layers
        self.conv1 = GCNConv(in_channels, hidden_dim)

```

```

self.conv2 = GCNConv(hidden_dim, hidden_dim)
self.conv3 = GCNConv(hidden_dim, hidden_dim)

self.bn1 = nn.BatchNorm1d(hidden_dim)
self.bn2 = nn.BatchNorm1d(hidden_dim)
self.bn3 = nn.BatchNorm1d(hidden_dim)

self.lin1 = nn.Linear(hidden_dim, hidden_dim)
self.lin2 = nn.Linear(hidden_dim, out_dim)
self.dropout = nn.Dropout(dropout)

def forward(self, x, edge_index, batch):
    # first graph convolution block
    x = self.conv1(x, edge_index)
    x = self.bn1(x)
    x = F.relu(x)
    x = self.dropout(x)

    # second graph convolution block
    x = self.conv2(x, edge_index)
    x = self.bn2(x)
    x = F.relu(x)
    x = self.dropout(x)

    # third graph convolution block
    x = self.conv3(x, edge_index)
    x = self.bn3(x)
    x = F.relu(x)

    # pool from atom space to molecule space
    x = global_mean_pool(x, batch)

    # readout MLP
    x = self.lin1(x)
    x = F.relu(x)
    x = self.dropout(x)
    x = self.lin2(x)
    return x

class Trainer:
    def __init__(self, model, lr = 1e-3, device = "cpu"):
        self.model = model.to(device)
        self.device = device
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr = lr)
        self.loss_function = torch.nn.MSELoss()

    def train_one_epoch(self, loader):
        self.model.train() # training 'mode' as per tutorial
        total_loss = 0.0

        for batch in loader:
            batch = batch.to(self.device)

            self.optimizer.zero_grad() # grad not needed as we evaluate
            pred = self.model(batch.x, batch.edge_index, batch.batch)
            loss = self.loss_function(pred, batch.y)

```

```

        loss.backward() # find the gradients
        self.optimizer.step() # update the weights

        total_loss += loss.item()

    return total_loss / len(loader) # average loss per batch

# used for testing phase
@torch.no_grad()
def evaluate(self, loader):
    self.model.eval()
    total_loss = 0.0
    all_preds = [] # Collect all predictions
    all_targets = [] # Collect all targets

    for batch in loader:
        batch = batch.to(self.device)
        pred = self.model(batch.x, batch.edge_index, batch.batch)
        loss = self.loss_function(pred, batch.y)
        total_loss += loss.item()
        all_preds.append(pred.cpu()) # Store predictions for R2 calculation
        all_targets.append(batch.y.cpu()) # Store targets for R2 calculation

    all_preds = torch.cat(all_preds, dim=0) # Concatenate all predictions
    all_targets = torch.cat(all_targets, dim=0) # Concatenate all targets

    # Compute average R2 across outputs (measure of accuracy for regression)
    ss_res = ((all_preds - all_targets) ** 2).sum(dim=0) # Sum of squared residuals
    ss_tot = ((all_targets - all_targets.mean(dim=0)) ** 2).sum(dim=0) # Sum of squared total
    r2_per_output = 1 - (ss_res / ss_tot) # R2 per output
    avg_r2 = r2_per_output.mean().item() # Average R2 across all outputs

    return total_loss / len(loader), avg_r2

def fit(self, train_loader, val_loader, epochs = 20, show_progress = True):
    train_losses = []
    val_losses = []
    val_r2s = [] # Track R2 history

    best_val = float("inf")
    best_train = None
    best_val_r2 = -float("inf") # Track best R2

    for epoch in range(1, epochs + 1):
        train_loss = self.train_one_epoch(train_loader)
        val_loss, val_r2 = self.evaluate(val_loader) # Now returns loss and R2

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        val_r2s.append(val_r2) # Append R2 for this epoch

        if val_loss < best_val:
            best_val = val_loss
            best_train = train_loss
            best_val_r2 = val_r2 # Update best R2

```

```

        if show_progress:
            print(f"Epoch {epoch}: train={train_loss:.4f}, val={val_loss:.4f}")

    return {
        "train_losses": train_losses,
        "val_losses": val_losses,
        "val_r2s": val_r2s, # Include R2 history
        "best_train_loss": best_train,
        "best_val_loss": best_val,
        "best_val_r2": best_val_r2 # Include best R2
    }

```

In [10]: *# this is the stucture from homework 7, I am going to appropriate for my need*
replace accuracy with the R² regression statistic

```

def KFoldCrossValidation(
    model_class,
    graph_dataset,
    k = 3,
    batch_size = 32,
    epochs = 20,
    device = "cpu",
    early_break = False
):

    kf = KFold(n_splits = k, shuffle = True, random_state=67)

    train_loss_list = []
    val_loss_list = []
    val_r2_list = [] # List to store best validation R2 for each fold
    all_train_losses = [] # List to store full training loss history for each fold
    all_val_losses = [] # List to store full validation loss history for each fold
    all_val_r2s = [] # List to store full validation R2 history for each fold

    for fold, (train_idx, val_idx) in enumerate(kf.split(graph_dataset)):
        print(f"\nFold {fold+1}")

        # split data
        train_graphs = [graph_dataset[i] for i in train_idx]
        val_graphs = [graph_dataset[i] for i in val_idx]

        # loaders
        train_loader = DataLoader(train_graphs, batch_size = batch_size, shuffle = True)
        val_loader = DataLoader(val_graphs, batch_size = batch_size, shuffle = False)

        # new model class each fold
        model = model_class()
        trainer = Trainer(model, device = device)

        # train
        results = trainer.fit(train_loader, val_loader, epochs = epochs, show_progress = True)

        train_loss_best = results["best_train_loss"]
        val_loss_best = results["best_val_loss"]
        val_r2_best = results["best_val_r2"] # Best validation R2 from this fold

```

```

train_loss_list.append(train_loss_best)
val_loss_list.append(val_loss_best)
val_r2_list.append(val_r2_best) # Append best R2 for final averaging
all_train_losses.append(results["train_losses"]) # Append full train loss history
all_val_losses.append(results["val_losses"]) # Append full val loss history
all_val_r2s.append(results["val_r2s"]) # Append full val R2 history

print(f"Best train loss: {train_loss_best:.4f}")
print(f"Best val loss: {val_loss_best:.4f}")
print(f"Best val R2: {val_r2_best:.4f}") # Print R2 for this fold

if early_break:
    break

# Compute average losses and R2 per epoch across folds
avg_train_losses = np.mean(np.array(all_train_losses), axis=0) # Average train loss
avg_val_losses = np.mean(np.array(all_val_losses), axis=0) # Average val loss
avg_val_r2s = np.mean(np.array(all_val_r2s), axis=0) # Average val R2 per epoch

# Plot the loss curves to show decreasing loss
plt.figure(figsize=(10, 6))
plt.plot(avg_train_losses, label='Average Train Loss')
plt.plot(avg_val_losses, label='Average Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Training and Validation Loss Curves (K-Fold Cross-Validation)')
plt.legend()
plt.grid(True)
plt.show()

# Plot the R2 curve as a measure of accuracy
plt.figure(figsize=(10, 6))
plt.plot(avg_val_r2s, label='Average Validation R2', color='green')
plt.xlabel('Epoch')
plt.ylabel('R2')
plt.title('Validation R2 Curve (K-Fold Cross-Validation)')
plt.legend()
plt.grid(True)
plt.show()

print("\nFinal results:")
print(f"Train loss: {np.mean(train_loss_list):.4f} ± {np.std(train_loss_list):.4f}")
print(f"Val loss: {np.mean(val_loss_list):.4f} ± {np.std(val_loss_list):.4f}")
print(f"Val R2: {np.mean(val_r2_list):.4f} ± {np.std(val_r2_list):.4f}")

```

```

In [12]: # the pain and suffering of a mac user
device = "mps" if torch.cuda.is_available() else "cpu"

KFoldCrossValidation(
    model_class=GNN,
    graph_dataset=graph_dataset,
    k=3,
    batch_size=32,
    epochs=100,
    device=device
)

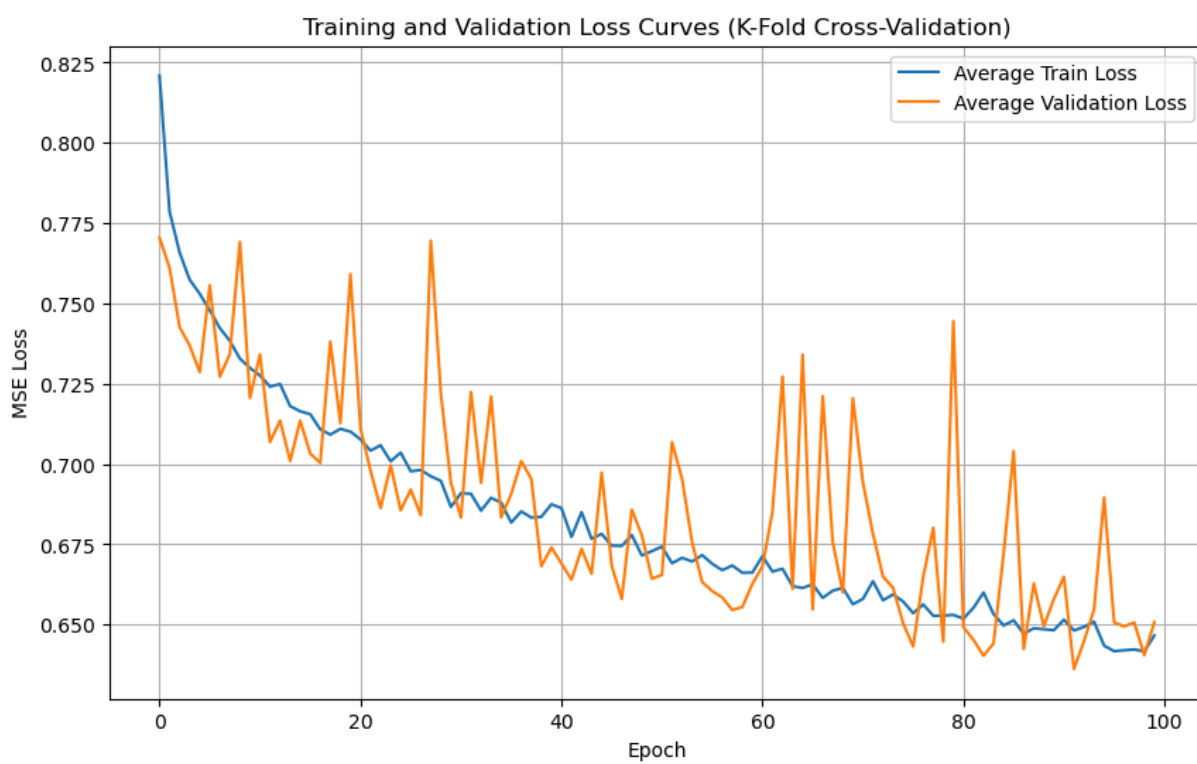
```

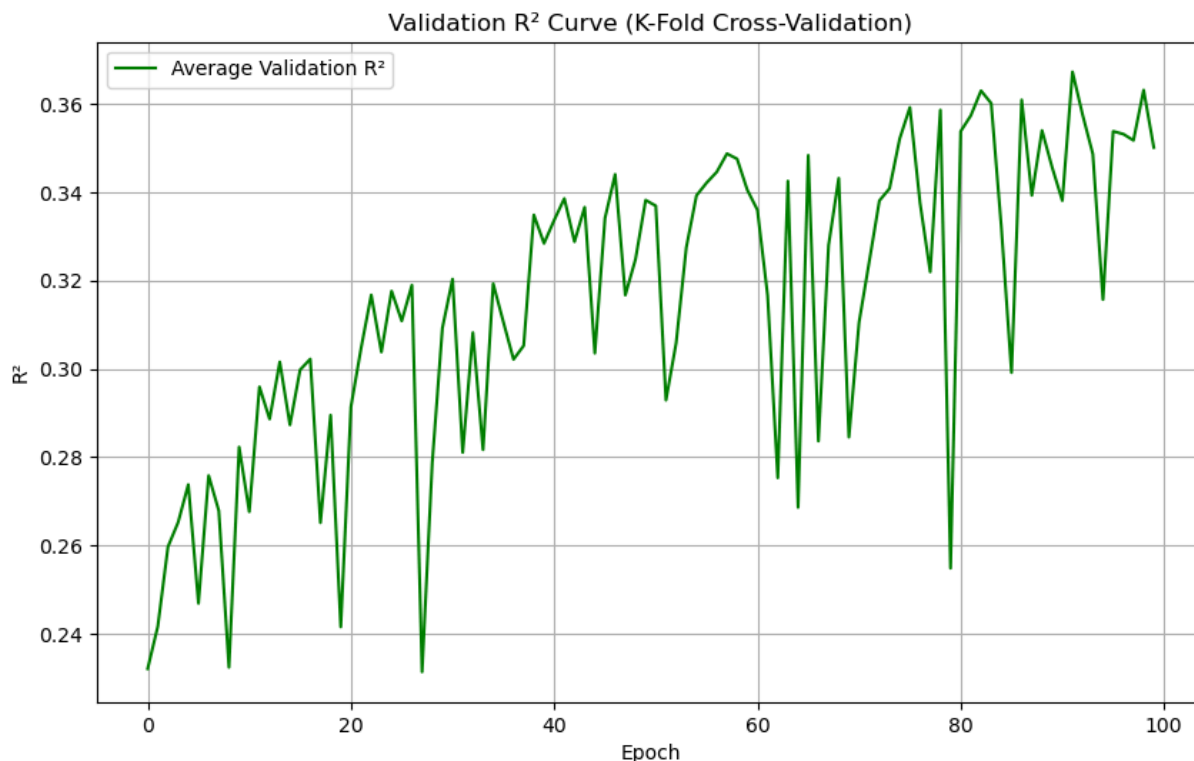
```
)  
# 4 minutes 22 sec with n_examples = 5
```

Fold 1
Best train loss: 0.6365
Best val loss: 0.6259
Best val R^2 : 0.3752

Fold 2
Best train loss: 0.6424
Best val loss: 0.6466
Best val R^2 : 0.3719

Fold 3
Best train loss: 0.6431
Best val loss: 0.6173
Best val R^2 : 0.3709





Final results:

Train loss: 0.6407 ± 0.0030

Val loss: 0.6299 ± 0.0123

Val R²: 0.3727 ± 0.0018

(2.1) Side code, not as relevant: PCA on target features and K-fold comparison

Run PCA on the 9 target features, build a new graph dataset with the reduced PCA targets, and compare the K-fold results to the original model. Result: Does not really help with the noise, but deal with that later in Checkpoint #3.

```
In [13]: from sklearn.decomposition import PCA

# Select PCA components to keep most of the target variance
n_pca_components = 9

# Original target matrix from the sampled molecules
target_matrix = small_molecule_types_df[molecule_targets_df.columns].astype(float)

pca = PCA(n_components=n_pca_components, random_state=67)
target_pca = pca.fit_transform(target_matrix)

print("Explained variance ratio (cumulative):", np.cumsum(pca.explained_variance_ratio_))
print("Total explained variance by first", n_pca_components, "components:", np.sum(pca.explained_variance_ratio_))

# Normalize PCA targets before using them for training
pca_mean = target_pca.mean(axis=0)
pca_std = target_pca.std(axis=0).clip(min=1e-8)
target_pca_norm = (target_pca - pca_mean) / pca_std
```

```
# Build a new graph dataset with PCA targets
graph_dataset_pca = []
for idx, row in small_molecule_types_df.iterrows():
    graph = smiles_to_graph(row['smiles'], target_pca_norm[idx])
    if graph is not None:
        graph_dataset_pca.append(graph)

print("PCA graph dataset size:", len(graph_dataset_pca))

# Run K-fold cross-validation on the PCA target dataset
KFoldCrossValidation(
    model_class=lambda: GNN(in_channels=1, hidden_dim=128, out_dim=n_pca_comps),
    graph_dataset=graph_dataset_pca,
    k=3,
    batch_size=32,
    epochs=100,
    device=device
) # 3 minutes 53 secs with PCA
```

Explained variance ratio (cumulative): [0.91708518 0.99956355 0.99998663 0.99999996 0.99999999 1.

1. 1. 1.]

Total explained variance by first 9 components: 0.9999999999999999

PCA graph dataset size: 5000

Fold 1

Best train loss: 0.6571

Best val loss: 0.6089

Best val R²: 0.3827

Fold 2

Best train loss: 0.6275

Best val loss: 0.6384

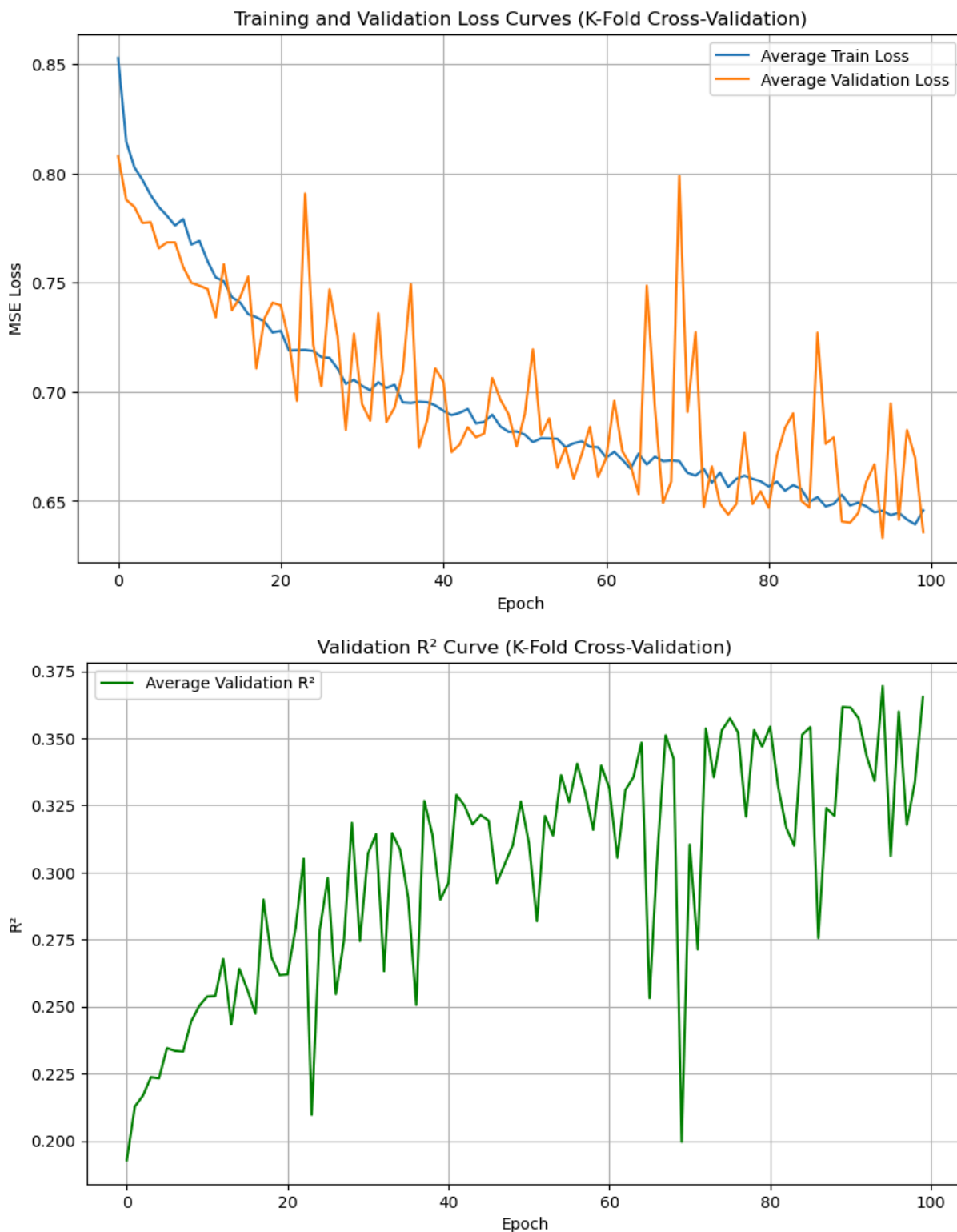
Best val R²: 0.3735

Fold 3

Best train loss: 0.6478

Best val loss: 0.6238

Best val R²: 0.3773



Final results:

Train loss: 0.6441 ± 0.0123

Val loss: 0.6237 ± 0.0120

Val R²: 0.3778 ± 0.0037

(3) Decision tree classification from GNN features

Use the trained GNN to predict molecular properties for the reactant and product SMILES, then train a simple decision tree to classify the reaction category. This will not

likely be part of the final code, but it does give me insight whether this GNN data is helpful at all whatsoever.

```
In [14]: from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.model_selection import StratifiedKFold

def smiles_to_graph_no_target(smi, out_dim=9):
    mol = Chem.MolFromSmiles(smi)
    if mol is None:
        return None

    atom_features = []
    for atom in mol.GetAtoms():
        atom_features.append([atom.GetAtomicNum()])
    x = torch.tensor(atom_features, dtype=torch.float)

    edge_index = []
    for bond in mol.GetBonds():
        i = bond.GetBeginAtomIdx()
        j = bond.GetEndAtomIdx()
        edge_index.append((i, j))
        edge_index.append((j, i))

    if len(edge_index) == 0:
        edge_index = torch.empty((2, 0), dtype=torch.long)
    else:
        edge_index = torch.tensor(edge_index, dtype=torch.long).t().contiguous()

    y = torch.zeros(out_dim, dtype=torch.float)
    return Data(x=x, edge_index=edge_index, y=y)

def predict_gnn_features(model, smi):
    graph = smiles_to_graph_no_target(smi, out_dim=9)
    if graph is None:
        return None
    graph = graph.to(device)
    batch = torch.zeros(graph.x.size(0), dtype=torch.long, device=device)
    model.eval()
    with torch.no_grad():
        return model(graph.x, graph.edge_index, batch).cpu().numpy()

# make a model and try out using features of GNN
feature_model = GNN(in_channels=1, hidden_dim=128, out_dim=9, dropout=0.2).to(device)
feature_trainer = Trainer(feature_model, lr=1e-3, device=device)

feature_loader = DataLoader(graph_dataset, batch_size=32, shuffle=True)
feature_trainer.fit(feature_loader, feature_loader, epochs=50, show_progress=True)

# predict the features of the molecules in the reaction scheme
reaction_features = []
reaction_labels = []

for _, row in reaction_types_df.iterrows():
    reactant_feat = predict_gnn_features(feature_model, row['reactant'])
```

```
product_feat = predict_gnn_features(feature_model, row['product'])
if reactant_feat is None or product_feat is None:
    continue
reaction_features.append(np.concatenate([reactant_feat, product_feat], a
reaction_labels.append(row['category'])

reaction_features = np.vstack(reaction_features)
reaction_labels = np.array(reaction_labels, dtype=int)

print('Reaction examples:', reaction_features.shape)
print('Reaction categories:', np.unique(reaction_labels))

# get the classification report for the tree
clf = DecisionTreeClassifier(max_depth=6, random_state=67)
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=67)

accuracies = []
for fold, (train_idx, test_idx) in enumerate(skf.split(reaction_features, re
    clf.fit(reaction_features[train_idx], reaction_labels[train_idx])
    preds = clf.predict(reaction_features[test_idx])
    acc = accuracy_score(reaction_labels[test_idx], preds)
    accuracies.append(acc)
    print(f'Fold {fold} accuracy: {acc:.4f}')
    print(classification_report(reaction_labels[test_idx], preds, zero_divis

print(f'Average decision tree accuracy: {np.mean(accuracies):.4f} ± {np.std(
# run time for n_examples = 5000 is 2m 30s
```

Reaction examples: (35116, 18)

Reaction categories: [1 2 3 4 5 6 7 8 9 10]

Fold 1 accuracy: 0.4699

	precision	recall	f1-score	support
1	0.58	0.34	0.43	2879
2	0.41	0.73	0.53	3577
3	0.00	0.00	0.00	290
4	0.00	0.00	0.00	202
5	0.80	0.13	0.23	178
6	0.53	0.64	0.58	2518
7	0.48	0.19	0.27	1398
8	0.00	0.00	0.00	244
9	0.00	0.00	0.00	411
10	0.00	0.00	0.00	9
accuracy			0.47	11706
macro avg	0.28	0.20	0.20	11706
weighted avg	0.45	0.47	0.43	11706

Fold 2 accuracy: 0.4795

	precision	recall	f1-score	support
1	0.51	0.43	0.47	2878
2	0.44	0.69	0.53	3577
3	0.00	0.00	0.00	291
4	0.00	0.00	0.00	202
5	0.59	0.11	0.18	177
6	0.55	0.62	0.58	2519
7	0.43	0.25	0.31	1398
8	0.00	0.00	0.00	244
9	0.00	0.00	0.00	411
10	0.00	0.00	0.00	8
accuracy			0.48	11705
macro avg	0.25	0.21	0.21	11705
weighted avg	0.44	0.48	0.44	11705

Fold 3 accuracy: 0.4800

	precision	recall	f1-score	support
1	0.58	0.36	0.45	2878
2	0.42	0.77	0.54	3577
3	0.00	0.00	0.00	290
4	0.00	0.00	0.00	202
5	0.58	0.10	0.17	178
6	0.57	0.60	0.59	2519
7	0.45	0.21	0.29	1397
8	0.00	0.00	0.00	244
9	0.00	0.00	0.00	411
10	0.00	0.00	0.00	9
accuracy			0.48	11705
macro avg	0.26	0.20	0.20	11705
weighted avg	0.46	0.48	0.44	11705

Average decision tree accuracy: 0.4764 ± 0.0046

50% is not bad, considering we used here like 2.5% of the molecule data.

(3.1) Other Idea: Use a Transformer instead of a decision tree, with and without the information gained by the GNN.

Set up: Reactants is one side, products is the otherside. We want to translate it, also adding in info from the GNN somewhere.

```
In [ ]: from torch.nn import TransformerEncoder, TransformerEncoderLayer
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

# SMILES Tokenizer (character-level)
class SmileTokenizer:
    def __init__(self):
        """
        0: pad shorter sequences
        1: unknown character
        2: seperator between reactants and products
        """
        self.char2idx = {'[PAD]': 0, '[UNK]': 1, '[SEP]': 2}
        self.idx2char = {0: '[PAD]', 1: '[UNK]', 2: '[SEP]'}
        self.n_chars = 3 # how many tokens are in library

    # add character from smiles string into vocabulary
    def add_smile(self, smile):
        for char in smile:
            if char not in self.char2idx:
                self.char2idx[char] = self.n_chars
                self.idx2char[self.n_chars] = char
                self.n_chars += 1

    # turn smile into a list of numbers else 1 which is [UNK]
    def tokenize(self, smile):
        return [self.char2idx.get(char, 1) for char in smile]

    # token IDs back to characters
    def detokenize(self, tokens):
        return ''.join(self.idx2char.get(t, '[UNK]') for t in tokens)

    # Use a small sampled dataset for a quick test run
    sample_size = 1000
    reaction_types_df_small = reaction_types_df.sample(n=min(sample_size, len(reaction_types_df)))

    # Initialize tokenizer
    smile_tok = SmileTokenizer()
    for _, row in reaction_types_df_small.iterrows():
        smile_tok.add_smile(row['reactant'])
```

```

smile_tok.add_smile(row['product'])

# Encode reaction categories as 0, 1, 2, ... for CrossEntropyLoss
# Learned this was just a fancy way of easily converting text labels into nu
label_encoder = LabelEncoder()
reaction_types_df_small = reaction_types_df_small.copy()
reaction_types_df_small['category_encoded'] = label_encoder.fit_transform(re

# Dataset
class ReactionDataset(Dataset):
    def __init__(self, df, tokenizer):
        self.data = []
        for _, row in df.iterrows():
            reactant_tokens = tokenizer.tokenize(row['reactant'])
            product_tokens = tokenizer.tokenize(row['product'])
            combined = reactant_tokens + [tokenizer.char2idx['[SEP]']] + pro
            self.data.append((torch.tensor(combined, dtype=torch.long), row)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

# Collate function for padding
# combine multiple examples into a batch
def collate_fn(batch):
    inputs = [b[0] for b in batch]
    labels = torch.tensor([b[1] for b in batch], dtype=torch.long)
    inputs_padded = pad_sequence(inputs, padding_value=0, batch_first=True)
    return inputs_padded, labels

# Transformer Classifier Model
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, emb_size, nhead, num_layers, num_classes,
                 dropout):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, emb_size, padding_idx=0)
        encoder_layer = TransformerEncoderLayer(emb_size, nhead, dim_feedfor
        self.transformer = TransformerEncoder(encoder_layer, num_layers)
        self.classifier = nn.Linear(emb_size, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.embedding(x)
        x = self.transformer(x)
        x = x.mean(dim=1) # Global average pooling
        x = self.dropout(x)
        return self.classifier(x)

# Model parameters
vocab_size = smile_tok.n_chars
emb_size = 64
nhead = 4
num_layers = 2
num_classes = len(label_encoder.classes_)

```

```

model = TransformerClassifier(vocab_size, emb_size, nhead, num_layers, num_c

# Training function
def train_transformer(model, train_loader, epochs=10, lr=1e-3):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss() # for classification tasks
    model.train()
    for epoch in range(epochs):
        total_loss = 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = loss_fn(outputs, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        print(f'Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}')

# Evaluation
def evaluate_transformer(model, test_loader):
    model.eval()
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    return accuracy_score(all_labels, all_preds)

# K-Fold Cross Validation
# Use only the sampled subset here so this can finish quickly.
dataset = ReactionDataset(reaction_types_df_small, smile_tok)
skf = StratifiedKFold(n_splits=2, shuffle=True, random_state=67)
accuracies = []
batch_size = 64
epochs = 5

for fold, (train_idx, test_idx) in enumerate(skf.split(range(len(dataset)),
train_subset = torch.utils.data.Subset(dataset, train_idx)
test_subset = torch.utils.data.Subset(dataset, test_idx)
train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_subset, batch_size=batch_size, shuffle=False)

    model = TransformerClassifier(vocab_size, emb_size, nhead, num_layers, n
    train_transformer(model, train_loader, epochs=epochs)
    acc = evaluate_transformer(model, test_loader)
    accuracies.append(acc)
    print(f'Fold {fold+1} Accuracy: {acc:.4f}')

print(f'Average Transformer Accuracy: {np.mean(accuracies):.4f} ± {np.std(accuracies):.4f}')

```

Epoch 1, Loss: 1.8362
Epoch 2, Loss: 1.6986
Epoch 3, Loss: 1.6839
Epoch 4, Loss: 1.6404
Epoch 5, Loss: 1.6211
Fold 1 Accuracy: 0.3780
Epoch 1, Loss: 1.8135
Epoch 2, Loss: 1.6957
Epoch 3, Loss: 1.6490
Epoch 4, Loss: 1.6552
Epoch 5, Loss: 1.6369
Fold 2 Accuracy: 0.3600
Average Transformer Accuracy: 0.3690 $\pm$ 0.0090

This is a proof of concept, as this takes actually forever to work. Now I have to figure out how to shove in the GNN output information, likely as another token, but that will be left to be done by next checkpoint.

REALIZATION: Wait a second, frick, Cl as a token splits into C and l. Next time, write code where upon detecting l it pairs and identifies Cl for Chlorine.